

LAB 2

ML PIPELINE & EXPERIMENT TRACKING

Building Reproducible ML Pipelines with MLflow and Airflow

Course	DDM501 - AI in Production: From Models to Systems
Session	Session 5
Weight	15% of total grade
Format	Team Lab (3-4 members per team)
Prerequisites	Lab 1 completed

1. OVERVIEW

1.1. Introduction

In this lab, you will transform the movie rating prediction system from Lab 1 into a production-ready ML pipeline with experiment tracking and workflow orchestration. This is a critical step in MLOps - moving from ad-hoc model training to systematic, reproducible pipelines.

You will learn to track experiments systematically using MLflow, version your models, and automate the entire training workflow using Apache Airflow. These skills are essential for any ML Engineer working in production environments.

1.2. Learning Objectives

After completing this lab, students will be able to:

1. Build modular, reproducible ML pipelines
2. Track experiments systematically with MLflow (parameters, metrics, artifacts)
3. Version and register models using MLflow Model Registry
4. Create and schedule Airflow DAGs for pipeline orchestration
5. Compare experiments and select the best model
6. Automate retraining workflows

1.3. Connection to Theory

This lab applies knowledge from the following theory sessions:

- **Session 4:** ML Pipelines - Pipeline architecture and design patterns
- **Session 5:** Experiment Tracking & Model Versioning - MLflow concepts

1.4. Scenario: Scaling the ML System

Your movie rating prediction API from Lab 1 is now in production. The Data Science team wants to:

- Experiment with different model architectures (SVD, NMF, KNN)
- Track all experiments to compare results systematically
- Automate weekly model retraining with new data
- Version models and rollback if new models perform worse

Your task is to build the MLOps infrastructure to support these requirements.

2. BACKGROUND KNOWLEDGE

2.1. ML Pipeline Architecture

A typical ML pipeline consists of sequential stages, each with specific responsibilities:

Stage	Description
Data Ingestion	Load raw data from sources (databases, files, APIs)
Data Validation	Check data quality, schema, and statistics
Data Preprocessing	Clean, transform, and normalize data
Feature Engineering	Create and select features for model training
Model Training	Train model with hyperparameter tuning
Model Evaluation	Evaluate model performance on test data
Model Registry	Version and register model for deployment

2.2. MLflow Overview

MLflow is an open-source platform for managing the ML lifecycle. It has four main components:

- **MLflow Tracking:** Log parameters, metrics, and artifacts during experiments
- **MLflow Projects:** Package ML code in a reusable, reproducible format
- **MLflow Models:** Deploy models to various serving platforms
- **MLflow Registry:** Centralized model store with versioning and stage management

Key MLflow Concepts:

- **Experiment:** Collection of runs for a specific task
- **Run:** Single execution of training code
- **Parameters:** Input configuration (hyperparameters)
- **Metrics:** Output measurements (RMSE, MAE, accuracy)
- **Artifacts:** Output files (models, plots, data)

2.3. Apache Airflow Overview

Apache Airflow is a platform to programmatically author, schedule, and monitor workflows. Key concepts:

- **DAG (Directed Acyclic Graph):** Collection of tasks with dependencies
- **Task:** Single unit of work (Python function, Bash command)
- **Operator:** Template for tasks (PythonOperator, BashOperator)
- **Schedule:** Cron expression defining when DAG runs

2.4. Why MLflow + Airflow?

Aspect	MLflow	Airflow
Primary Purpose	Experiment tracking & model registry	Workflow orchestration
Focus	ML-specific (parameters, metrics)	General task scheduling
Strengths	Model versioning, comparison UI	Complex dependencies, monitoring
Together	Complete MLOps solution: Airflow orchestrates pipeline, MLflow tracks experiments	

3. HANDS-ON GUIDE

Task 1: Setup MLflow Environment

1.1. Install MLflow

```
pip install mlflow==2.9.2
```

1.2. Start MLflow Tracking Server

```
# Start MLflow UI (in a separate terminal)
mlflow ui --host 0.0.0.0 --port 5000

# Or with backend store
mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlruns \
  --host 0.0.0.0 --port 5000
```

Access MLflow UI at: <http://localhost:5000>

1.3. Configure MLflow in Python

```
import mlflow
```

```
# Set tracking URI
mlflow.set_tracking_uri("http://localhost:5000")
# Create or set experiment
mlflow.set_experiment("movie-rating-prediction")
# Verify connection
print(f"Tracking URI: {mlflow.get_tracking_uri()}")
print(f"Experiment: {mlflow.get_experiment_by_name('movie-rating-prediction')}")
```

Task 2: Build Modular ML Pipeline (60 minutes)

2.1. Pipeline Structure

Create modular pipeline with separate stages:

```
pipeline/
├── __init__.py
├── config.py          # Configuration parameters
├── data_ingestion.py  # Load and split data
├── preprocessing.py   # Data preprocessing
├── training.py        # Model training with Mlflow
├── evaluation.py      # Model evaluation
├── registry.py        # Model registration
└── run_pipeline.py    # Orchestrate all stages
```

2.2. Data Ingestion Stage

```
# pipeline/data_ingestion.py
from surprise import Dataset
from surprise.model_selection
import train_test_split
import logging

logger = logging.getLogger(__name__)

def load_data(dataset_name: str = 'ml-100k'):
    """Load dataset from surprise built-in datasets."""
    logger.info(f>Loading dataset: {dataset_name}")
    data = Dataset.load_builtin(dataset_name)
    return data

def split_data(data, test_size: float = 0.2):
    """Split data into train and test sets."""
    logger.info(f>Splitting data with test_size={test_size}")
    trainset, testset = train_test_split(data, test_size=test_size)
    return trainset, testset
```

2.3. Training Stage with MLflow

```
# pipeline/training.py
import mlflow
from surprise import SVD, NMF, KNNBasic
import pickle
```

```

MODEL_CLASSES = {
    'svd': SVD,
    'nmf': NMF,
    'knn': KNNBasic
}

def train_model(trainset, model_type: str = 'svd', **params):
    """Train model and log to MLflow."""
    with mlflow.start_run():
        # Log parameters
        mlflow.log_param("model_type", model_type)
        for key, value in params.items():
            mlflow.log_param(key, value)
        # Initialize and train model
        model_class = MODEL_CLASSES[model_type]
        model = model_class(**params)
        model.fit(trainset)
        # Save model artifact
        with open("model.pkl", "wb") as f:
            pickle.dump(model, f)
            mlflow.log_artifact("model.pkl")
        return model, mlflow.active_run().info.run_id

```

2.4. Evaluation Stage

```

# pipeline/evaluation.py
import mlflow
from surprise import accuracy
import matplotlib.pyplot as plt

def evaluate_model(model, testset, run_id: str):
    """Evaluate model and log metrics to MLflow."""
    with mlflow.start_run(run_id=run_id):
        # Make predictions
        predictions = model.test(testset)
        # Calculate metrics
        rmse = accuracy.rmse(predictions, verbose=False)
        mae = accuracy.mae(predictions, verbose=False)
        # Log metrics
        mlflow.log_metric("rmse", rmse)
        mlflow.log_metric("mae", mae)
        # Create and log plots
        fig = create_prediction_plot(predictions)
        mlflow.log_figure(fig, "prediction_distribution.png")
        return {"rmse": rmse, "mae": mae}

```

Task 3: Experiment Tracking with MLflow

3.1. Run Multiple Experiments

```
# experiments/run_experiments.py
import mlflow

from pipeline.data_ingestion import load_data, split_data
from pipeline.training import train_model
from pipeline.evaluation import evaluate_model

# Define experiment configurations
EXPERIMENTS = [
    {"model_type": "svd", "n_factors": 50, "n_epochs": 20},
    {"model_type": "svd", "n_factors": 100, "n_epochs": 20},
    {"model_type": "svd", "n_factors": 100, "n_epochs": 50},
    {"model_type": "nmf", "n_factors": 50, "n_epochs": 50},
    {"model_type": "knn", "k": 40, "sim_options": {"name": "cosine"}},
]

def run_all_experiments():
    mlflow.set_experiment("movie-rating-hyperparameter-tuning")
    # Load data once
    data = load_data()
    trainset, testset = split_data(data)
    results = []
    for config in EXPERIMENTS:
        model_type = config.pop("model_type")
        model, run_id = train_model(trainset, model_type, **config)
        metrics = evaluate_model(model, testset, run_id)
        results.append({"config": config, "metrics": metrics})
    return results
```

3.2. Compare Experiments in MLflow UI

After running experiments, use MLflow UI to:

- Compare runs side-by-side
- Sort by metrics (RMSE, MAE)
- View parameter vs metric charts
- Download artifacts (models, plots)

3.3. Model Registry

```
# pipeline/registry.py
import mlflow

from mlflow.tracking import MlflowClient

def register_best_model(experiment_name: str, model_name: str):
    """Find best run and register model."""
    client = MlflowClient()
    # Get experiment
    experiment = client.get_experiment_by_name(experiment_name)
```

```

# Find best run by RMSE
runs = client.search_runs(
    experiment_ids=[experiment.experiment_id],
    order_by=["metrics.rmse ASC"],
    max_results=1
)
best_run = runs[0]
# Register model
model_uri = f"runs://{best_run.info.run_id}/model"
mlflow.register_model(model_uri, model_name)
# Transition to Production
client.transition_model_version_stage(
    name=model_name,
    version=1,
    stage="Production"
)
return best_run.info.run_id

```

Task 4: Airflow Pipeline Orchestration

4.1. Install and Setup Airflow

```

# Install Airflow
pip install apache-airflow==2.8.0
# Initialize database
airflow db init
# Create admin user
airflow users create \
    --username admin \
    --password admin \
    --firstname Admin \
    --lastname User \
    --role Admin \
    --email admin@example.com
# Start webserver (terminal 1)
airflow webserver --port 8080
# Start scheduler (terminal 2)
airflow scheduler

```

4.2. Create Training DAG

```

# dags/ml_training_dag.py
from airflow import DAG
from airflow.operators.python
import PythonOperator
from datetime import datetime, timedelta

```

```

default_args = {
    'owner': 'mlops',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}
dag = DAG(
    'movie_rating_training',
    default_args=default_args,
    description='ML Training Pipeline',
    schedule_interval='@weekly', # Run weekly
    catchup=False,
)
# Task definitions
load_data_task = PythonOperator(
    task_id='load_data',
    python_callable=load_data_fn,
    dag=dag,
)
train_model_task = PythonOperator(
    task_id='train_model',
    python_callable=train_model_fn,
    dag=dag,
)
evaluate_task = PythonOperator(
    task_id='evaluate_model',
    python_callable=evaluate_fn,
    dag=dag, )
register_task = PythonOperator(
    task_id='register_model',
    python_callable=register_fn,
    dag=dag,
)
# Define dependencies
load_data_task >> train_model_task >> evaluate_task >> register_task

```

4.3. DAG Task Functions

```

# Task functions with XCom for data passing
def load_data_fn(**context):
    """Load and prepare data."""
    from pipeline.data_ingestion import load_data, split_data
    data = load_data()

```



```

trainset, testset = split_data(data)
# Save to temporary location
import pickle
with open('/tmp/trainset.pkl', 'wb') as f:
    pickle.dump(trainset, f)
with open('/tmp/testset.pkl', 'wb') as f:
    pickle.dump(testset, f)
    return "Data loaded successfully"
def train_model_fn(**context):
    """Train model and log to MLflow."""
    import pickle
    from pipeline.training import train_model
    with open('/tmp/trainset.pkl', 'rb') as f:
        trainset = pickle.load(f)
        model, run_id = train_model(
            trainset,
            model_type='svd',
            n_factors=100,
            n_epochs=20
        )
        # Pass run_id to next task
        context['ti'].xcom_push(key='run_id', value=run_id)
    return run_id

```

4. STARTER CODE TEMPLATE

Clone starter repository:

```
Unzip ddm501-lab2-starter.git
```

Files to complete:

File	TODO Items
pipeline/training.py	Implement train_model() with MLflow logging
pipeline/evaluation.py	Implement evaluate_model() with metrics logging
pipeline/registry.py	Implement register_best_model()
dags/ml_training_dag.py	Create complete Airflow DAG with all tasks
experiments/run_experiments.py	Run hyperparameter tuning experiments
docker-compose.yml	Add MLflow and Airflow services

5. DELIVERABLES & GRADING

5.1. Deliverables

Submit GitHub repository link containing:

1. **Complete ML Pipeline:** Modular, reusable pipeline code
2. **MLflow Tracking Setup:** Configured experiment tracking with logged runs
3. **Airflow DAG:** Working DAG definition for training pipeline
4. **Experiment Report:** Comparison of at least 5 experiments with analysis

5. Documentation: README with setup and usage instructions

5.2. Grading Rubric

Criteria	Weight	Detailed Description
Pipeline Quality	35%	• Modular structure (10%) • Reproducible execution (10%) • Error handling (8%) • Code quality (7%)
Experiment Tracking	25%	• MLflow setup correct (8%) • Parameters logged (5%) • Metrics logged (5%) • Artifacts logged (4%) • Model registry (3%)
Airflow Automation	20%	• DAG structure correct (8%) • Tasks execute properly (7%) • Schedule configured (5%)
Documentation	20%	• Experiment report (10%) • README complete (5%) • Code documentation (5%)

5.3. Submission

- **Deadline:** 1 week after the lab session
- **Format:** GitHub repository link
- **Required:** Screenshots of MLflow UI showing experiments